

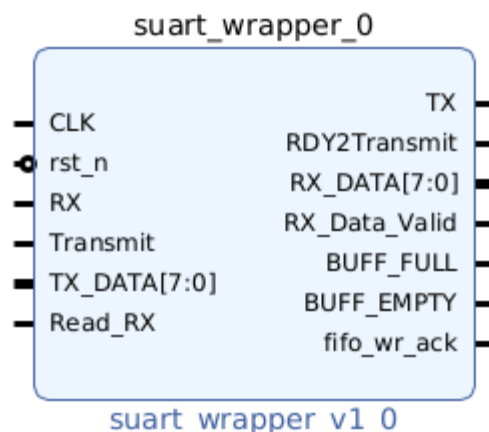
Super UART

Introduction:

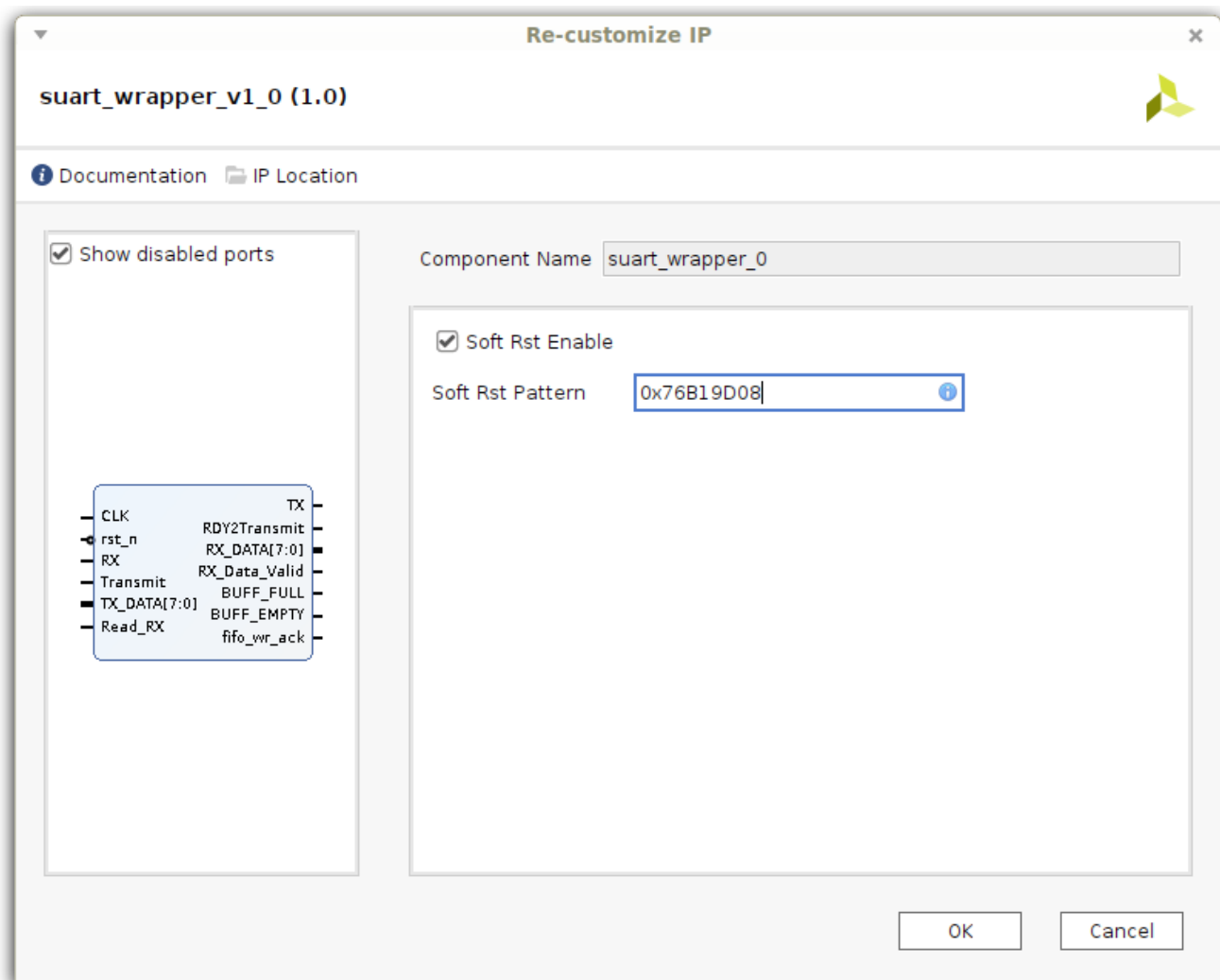
super uart is implemented in order to overcome all limitations of the ordinary uart designs. It has several features:

- 1- It uses the Xilinx FIFO ip BRAM for storing the RX data, so it can be configured for a desired length of data depends to the limitation of the device.
- 2- It is not only designed to accept or generate the ordinary baud rates, but also works for any user defined baud rate.
- 3- The accepted or generated baud rates are limited to $\frac{1}{4}$ of the main clock frequency. For instance if the main clock frequency is 100 MHz, then the maximum baud rate of 25 Mbps is possible if and only if the HW lets you to do so.
- 4- It has been tested for the minimum baud rate of as low as 300 to the maximum PC generated baud rate of 1843200 successfully.
- 5- It is in auto detection mode for locking itself to the master baud rate. It means that when it is in the initial state, user (PC) can send to it a message of "0x55" for initializing it which helps the ip to detect the baud rate of the transmitter.
- 6- If it is not in initial state (has been already used by user with a certain baud rate) it can accept a soft reset key or hard reset signal for starting with a new baud rate detection. By default the soft reset key is set to "0x76B19D08". There is a possibility to disable the soft reset key or modify it to a desired 32 bits key (These features are only available when using the source code, not the encrypted ip core).

The ip looks like this:



And usually when working with non-encrypted version of it, its gui should look like this:



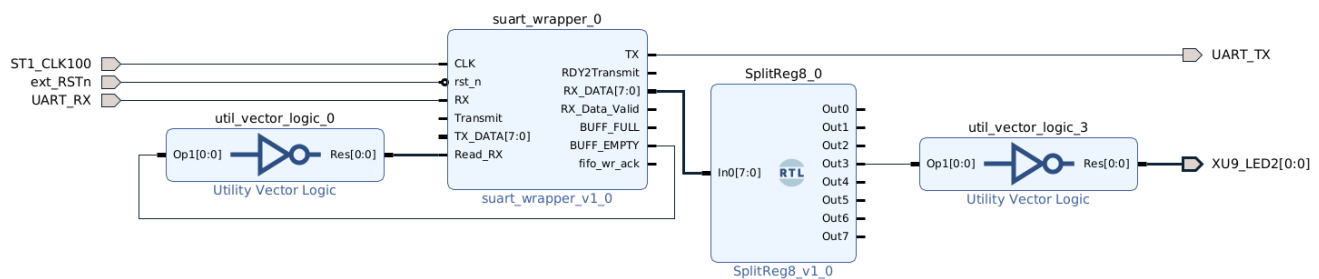
It is developed for no polarity check and 8 bit data, thereby the input and output values will be 8 bits registers.

Ports description:

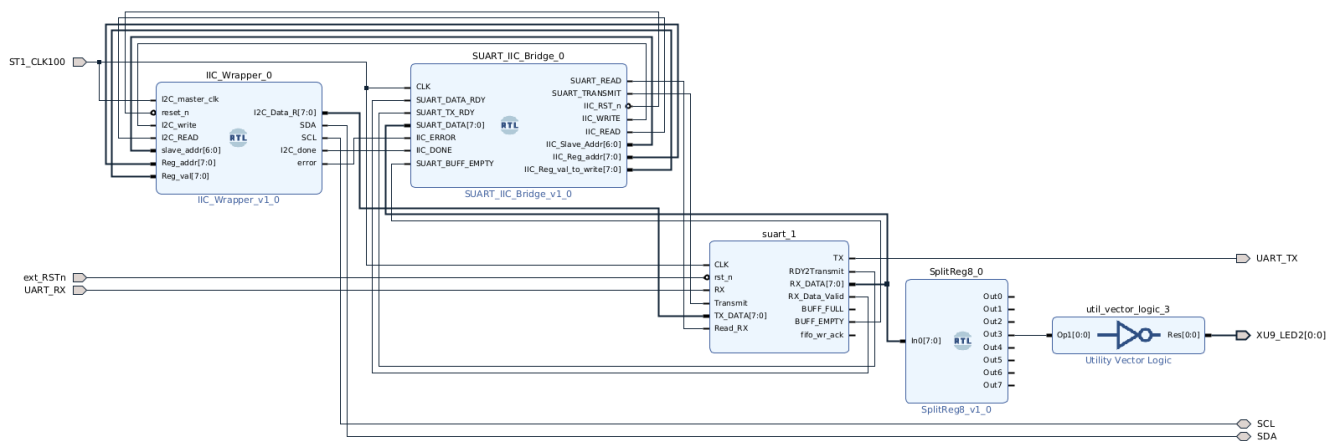
- CLK: This is the main input clock which must be connected to a clock signal with frequency of at least 4x of desired baud rate.
- rst_n: This is the HW active_low reset pin.
- RX: It is the UART RX (which should be pulled up, or if the master is already using pull-up then just connect it to master TX).
- Transmit: When RDY2Transmit signal is high, you can use this pin to transmit the 8 bit data presented (by you) at TX_DATA to the TX uart with locked baud rate. This signal only accepts rising edge (you should assert it when it is released).
- TX_DATA: This is used to transmit a desired byte to outside world. For this version of suart, we have not implemented and FIFO for TX part but it is possible per request.

- **Read_RX:** All data that are transmitted by outside world into RX, are registered into the rx FIFO. You can have access to them (read them) via RX_DATA port by asserting this signal. Same as Transmit, Read_RX also only accepts rising edge.
- **TX:** It should be tight to the RX of the master uart. The suart will transmit the data using this pin.
- **RDY2Transmit:** While suart is transmitting data over TX, it won't accept new data to transmit. Thereby RDY2Transmit will be "low" during transition. It will be asserted while the core is ready to accept and transmit new byte of data.
- **RX_Data_Valid:** While reading the receipted data from FIFO, you should wait till RX_Data_Valid pin be asserted. It shows that the data is fully validated and is ready on the RX_DATA port.
- **BUFF_FULL:** if by somehow the core gets too many data and its rx FIFO full, this pin will be asserted indicating that no more data is accepted via RX and there is a chance of losing incoming data.
- **BUFF_EMPTY:** This is the most useful pin of the core. If this pin is released it shows that at least there is one byte of data presented in the rx FIFO to be read. Usually we use this pin as an interrupt or indicator to read the data from the core.
- **fifo_wr_ack:** This pin indicated that the receipt data by the core is completely written into the FIFO.

Simple example design:



More complete example design:



Example Python code to interact with suart:

```
1
2
3  from time import sleep
4  import serial
5
6
7  FPGA = serial.Serial('/dev/ttyUSB0', 115200*16) #define your serial port and baudrate
8  FPGA.timeout = 1
9  FPGA.isOpen()
10     # print("Port not open.")
11     # exit()
12
13  def init():
14     # send pattern for SUART baud rate detection
15     FPGA.write(b'\x55')
16     sleep(0.1)
17
18  def reset():
19     #Set the command##
20     FPGA.write(b'\x76')
21     FPGA.write(b'\xB1')
22     FPGA.write(b'\x9D')
23     FPGA.write(b'\x08')
24     sleep(0.1)
25
```

```
81
82  init()
83
84  FPGA.write(b'\x08')
85  # iic_write(slave_add, 0x00, 0x00)
86  # iic_read(slave_add, 106)
87  reset()
88
89  FPGA.close()
90  exit()
91
```